STANFORD CS336 / SPRING 2026

Lecture 8

Parallelism Basics: 并行基础

从集合通信到 3D 并行：多 GPU / 多机训练大模型的结构化学习笔记

一句话主线

单卡“算不动、装不下”，于是把模型和 batch 切开分给多卡：DP 切 batch、PP 切深度、TP/SP 切宽度、EP 切专家；再按网络带宽分层放置——节点内 TP，跨节点 PP，剩余用 DP。

讲义主线：Part 1 网络与集合通信基础 → Part 2 并行原语（DP/ZeRO · 流水线 · 张量/序列 · 专家）→ Part 3 组合策略与真实训练案例

本笔记以 lectures/lecture_08.pdf（73 页，Tatsu Hashimoto）为基础，补充显存 / 通信量推导与工程判断。

学习笔记生成时间：2026-10-06 | 输出目录：output/pdf/

目录

01 为什么需要多 GPU 并行 4

1.1 单卡的两个墙 4

1.2 多机 scaling 的三个目标（Part 1 recap） 4

1.3 集合通信原语 4

1.4 网络拓扑：TPU vs GPU，以及 domain size 5

02 数据并行与 ZeRO 5

2.1 朴素数据并行（naïve DP） 5

2.2 显存到底要多少：16 字节 / 参数 5

2.3 ZeRO-1：分片优化器状态 6

2.4 ZeRO-2：再分片梯度 6

2.5 ZeRO-3 / FSDP：全分片 6

2.6 实战：8×A100 80G 能训多大? 7

2.7 数据并行的剩余问题 7

03 流水线并行 7

3.1 逐层并行（layer-wise）的问题 7

3.2 流水线并行（pipeline parallel） 8

3.3 为什么还要用流水线? 8

04 张量并行与序列并行 8

4.1 矩阵乘法的子矩阵分解 8

4.2 Transformer block 里的切法 9

4.3 什么时候用 TP? 9

4.4 TP vs PP 9

4.5 激活显存：34sbh + 5ash 9

4.6 序列并行（sequence parallel） 10

05 专家并行与其他并行 10

5.1 专家并行（expert parallel, EP） 10

5.2 EP 与其他并行的组合 10

5.3 解耦 attention / expert 并行（Megatron） 10

5.4 Context parallel / Ring attention 10

5.5 并行方法总表（slide 55） 10
06 组合策略与实战案例 11
6.1 3D（4D）并行：组合规则 11
6.2 经验法则 11
6.3 真实模型怎么配 12
6.4 案例总表（slide 72） 12
07 全讲回顾 13

01 为什么需要多 GPU 并行

第 8 讲回答“单卡不够之后，如何用多 GPU、多机训练大模型”。讲义分三部分：Part 1 是 LLM 的网络基础；Part 2 是不同形式的并行训练；Part 3 是如何用并行把大 LM 真正 scale 起来。

1.1 单卡的两个墙

- 算力墙（compute）：单 GPU scaling 有上限；世界最快的超算已达 exaflops 级，继续 scale 只能走向多 GPU / 多机。
- 显存墙（memory）：模型越来越大，单卡装不下大多数大模型。
- 出路：intra-node 高速互联 + 高速 inter-node 网络，把内存与算力需求切分到 GPU 和机器之间。

1.2 多机 scaling 的三个目标（Part 1 recap）

- 数据中心成为新的计算单元（new unit of compute – the datacenter）。
- 线性显存扩展：最大可训参数量随 GPU 数线性增长。
- 线性算力扩展：模型 flops 随 GPU 数线性增长。
- 只需要少数简单的集合通信原语。

1.3 集合通信原语

多卡协作的最小操作单元是集合通信（collective communication）。讲义给出 5 个原语：

原语	输入	输出
all-reduce	每个 rank 有 x_i	所有 rank 得到 Σx_i
all-gather	每个 rank 有 1/W 分片	所有 rank 得到完整向量
reduce-scatter	每个 rank 有完整向量	每个 rank 得到 1/W 的求和
broadcast	rank 0 有数据	所有 rank 得到该数据
reduce	每个 rank 有 x_i	rank 0 得到 Σx_i

关键细节

all-reduce 可以拆成 reduce-scatter + all-gather 两步；在带宽受限区间，这就是最优实现——NCCL 等库正是这样实现的。

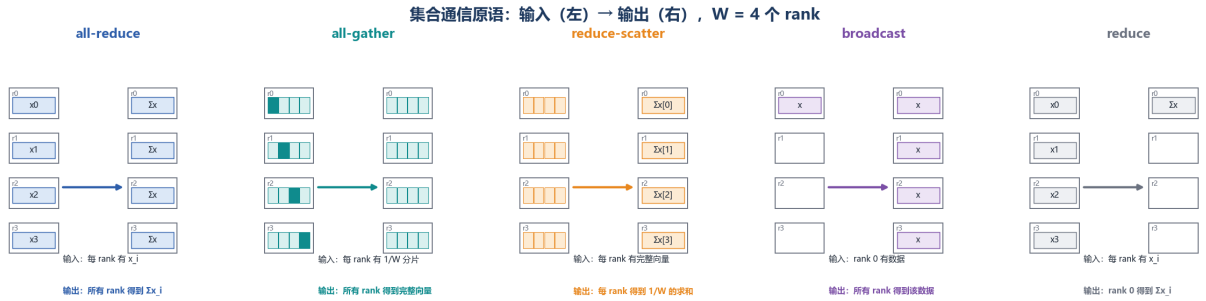


图 1 五个集合通信原语：输入（左）→ 输出（右），W = 4 个 rank

1.4 网络拓扑：TPU vs GPU，以及 domain size

- TPU：toroidal mesh（环面网格）；GPU：all-to-all（最多 256 卡，如 NVLink 域）。
- 为什么 mesh：便宜、易运维、可以做得很快，适合结构化通信（tensor parallel）。
- 为什么 tree / A2A：适合非结构化通信（expert parallel 的 all-to-all）。
- 新变化：TPU8i 网络更接近 tree 拓扑（可能为 MoE 服务）；TPU8t scale-out 用 “Virgo” 交换网络。
- Domain size：为什么不把所有卡连在一起？成本、可靠性与故障域是硬约束（讲义引用 SemiAnalysis 对华为 CloudMatrix 384 的分析）。

02 数据并行与 ZeRO

并行原语的第一类是数据并行。起点是朴素 SGD：$\theta_{t+1} = \theta_t - \eta \cdot \Sigma_{i=1..B} \text{ grad } f(x_i)$。

2.1 朴素数据并行（naïve DP）

- 做法：把 B 大小的 batch 切到 M 台机器，各自算梯度，交换梯度同步。
- 算力扩展：每卡 B/M 个样本，线性。
- 通信开销：每个 batch 传 2×#params（梯度 all-reduce）；batch 够大时 OK。
- 显存扩展：没有——每卡至少要有完整 #params。

2.2 显存到底要多少：16 字节 / 参数

按精度算，训练状态需要“5 份权重”，合计 16 B/param（不含激活）：

状态	精度	字节 / 参数
模型参数	BF16	2
梯度	BF16	2
master 权重（SGD 累加器）	FP32	4
Adam 一阶矩 m	FP32（或 BF16）	4（2）
Adam 二阶矩 v	FP32（或 BF16）	4（2）

状态	精度	字节 / 参数
合计	—	16

2.3 ZeRO-1：分片优化器状态

ZeRO 的核心思想：把贵的状态切开，利用 reduce-scatter ≡ all-reduce 的等价性。

- 做法：优化器状态（m/v）分片；参数和梯度每卡都有；每个 worker 负责更新自己那片参数。
- 流程：① 每卡在自己 batch 子集上算完整梯度；② reduce-scatter 梯度（通信 #params）；③ 每卡用自己的梯度 + 状态更新自己的参数片；④ all-gather 参数（通信 #params）。

	朴素 DDP	ZeRO-1
通信原语	一次 all-reduce（梯度）	一次 reduce-scatter（梯度）+ all-gather（参数）
通信代价	2×#params	2×#params
显存	(4+K)×#params	(4+K/N)×#params

结论
带宽受限区间 ZeRO-1 是“免费”的：通信量不变，显存直接降——应该总是开。

2.4 ZeRO-2：再分片梯度

- 梯度也分片；永远不实例化完整梯度向量，但每卡必须算完整梯度（因为是数据并行）。
- 增量技巧：反向逐层走——某层梯度算完立即 reduce 发给负责该片的 worker；梯度不再被反向图需要时立即释放。
- 通信 ≈ 2×#params，几乎免费。

2.5 ZeRO-3 / FSDP：全分片

- 连参数也分片；按计算图走到哪、就 on-demand 请求 / 发送参数，用完即释放。
- 通信：2 次 all-gather（前向参数、反向参数）+ 1 次 reduce-scatter（梯度）= 3×#params。
- 两个关键工程点：增量计算 / 通信；通信与计算重叠（all-gather 与 forward 同时进行，掩盖通信代价）。
- 概念上非常简单：写一个 FSDP block wrapper 即可。

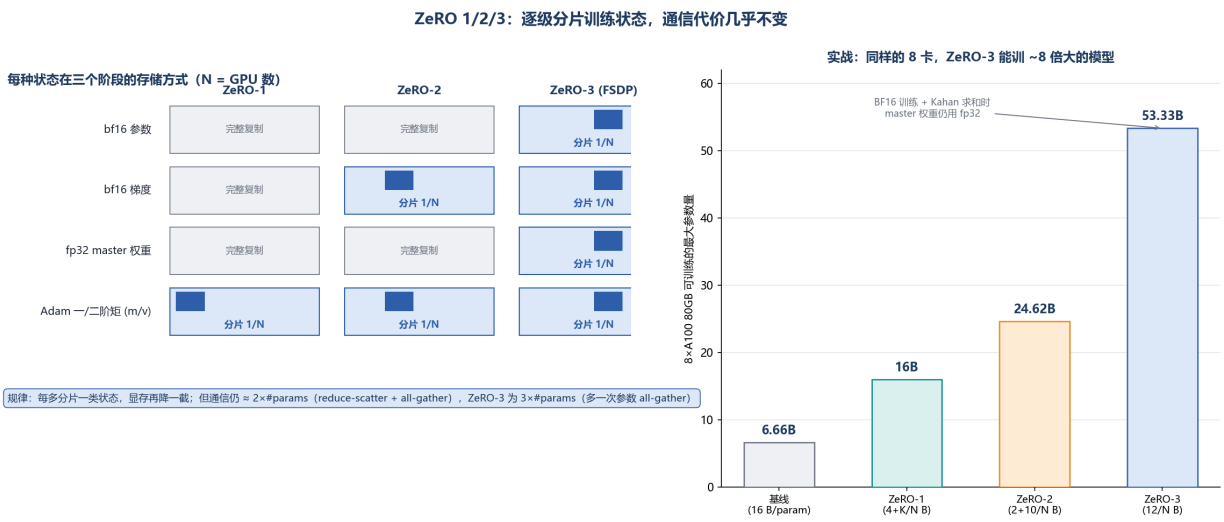


图 2 ZeRO 1/2/3 逐级分片训练状态，通信代价几乎不变（右：8×A100 80G 可训最大参数量）

2.6 实战：8×A100 80G 能训多大?

前提：纯 BF16 训练（Kahan 求和保证精度），master 权重仍用 FP32 → 12 B/param。

方案	最大参数量	B/param 公式
基线	6.66B	12
ZeRO-1	16B	5
ZeRO-2	24.62B	2 (param) + 10 (grad+state) / 8
ZeRO-3	53.33B	12 / 8

结论
同样的 8 卡，ZeRO-3 能训约 8 倍大的模型。

2.7 数据并行的剩余问题

- 算力扩展：#machines < batch size 才有效（接近时通信开销高），且 batch size 收益递减。
- 模型仍装不下：ZeRO-1/2 不 scale 显存；ZeRO-3 不减少激活显存。
- 结论：需要更好的“切模型”方式 → 模型并行。

03 流水线并行

模型并行 = 把参数切到多卡（像 ZeRO-3），但通信的是激活（ZeRO-3 传的是参数）。讲义覆盖三种切法：流水线（深度）、张量（宽度）、专家（MoE）。

3.1 逐层并行（layer-wise）的问题

- 把层切开分给 GPU，激活和梯度来回传。
- 利用率极差：n 张卡，每卡只有 1/n 时间在干活，其余时间等反向传回来。

3.2 流水线并行（pipeline parallel）

- 方案：micro-batch。发出第一个 micro-batch 后立刻算第二个，流水线填满。
- 气泡比例 = (n_stages − 1) / n_micro → 需要大 batch。

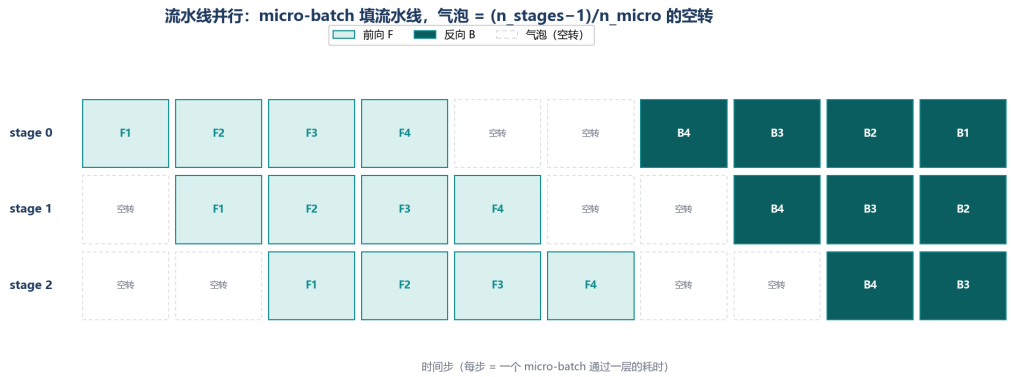


图 3 流水线并行：micro-batch 填流水线，气泡 = (n_stages−1)/n_micro 的空转

3.3 为什么还要用流水线?

- 比 DDP 省显存。
- 通信性质好：只依赖激活 b×s×h，且是点对点（point-to-point）。
- 一般用在慢链路上（inter-node），换取更好的显存扩展。
- 性能高度依赖 batch size：batch 太小气泡藏不住，性能快速退化。
- 更复杂的调度模式可以用带宽换利用率； “zero-bubble” 思路：把反向拆成 ① 回传激活梯度 (z, x) ② 算权重梯度 (W)，第二部分可以随时做。

04 张量并行与序列并行

4.1 矩阵乘法的子矩阵分解

- 观察：大矩阵乘法可以拆成子矩阵，部分和相加。
- 把列 (A1, A2) 和行 (B1, B2) 分给不同 GPU：前向 f 是恒等、g 是 all-reduce；反向 f 是 all-reduce、g 是恒等。

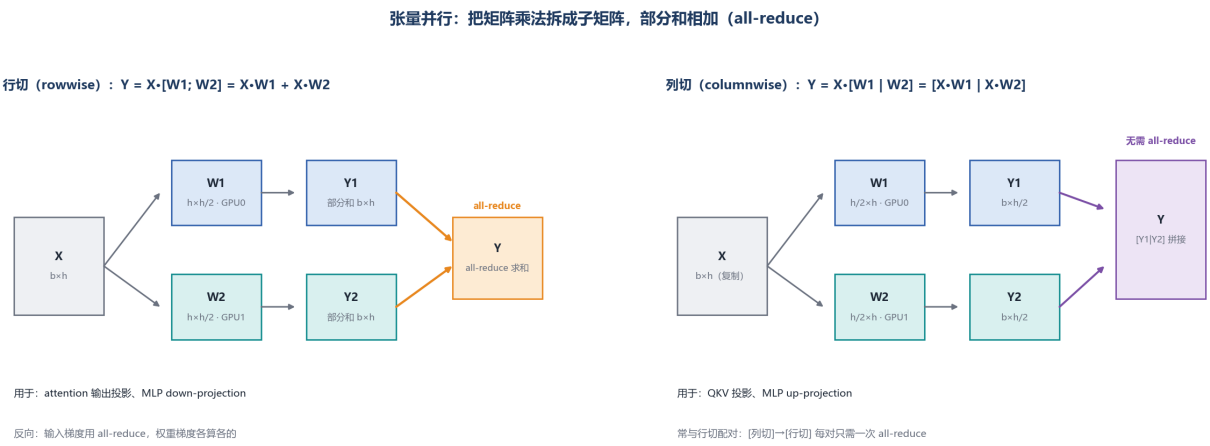


图 4 张量并行：把矩阵乘法拆成子矩阵，部分和相加（all-reduce）

4.2 Transformer block 里的切法

- Columnwise（列切）：QKV 投影、MLP up-projection。
- Rowwise（行切）：attention 输出投影、MLP down-projection。
- Replicated（复制）：norms、routers 等。
- 列切 → 行切配对：每对只需一次 all-reduce。

4.3 什么时候用 TP?

- GPU 上：节点内（最多 8 卡），因为高速互联（NVLink）。

4.4 TP vs PP

	流水线并行 PP	张量并行 TP
通信量	b×s×h 点对点 / micro-batch	8b×s×h×(n−1)/n 每层 all-reduce
气泡	有，(n_stages−1)/n_micro	无
batch 要求	需要大 batch	不依赖大 batch
复杂度	难	低（简单 wrap 模型）
适用链路	慢链路（inter-node）	低延迟高带宽（节点内）

原则
有低延迟、高带宽互联就用 TP。

4.5 激活显存：34sbh + 5ash

- 显存不只是参数，还有激活（动态、可能很大）。[Korthikanti et al 2022]
- 全存所有激活：每层约 34sbh + 5ash；5ash 来自二次 attention 项（含 dropout）。

- 像 FlashAttention 一样，用重计算（recomputation）丢掉 5ash。
- TP 切掉 attention + MLP 的矩阵乘法后，剩 10sbh：LayerNorm (4sbh) + Dropout (2sbh) + attention/MLP 输入 (4sbh)——这部分随模型规模继续增长。

4.6 序列并行（sequence parallel）

- 观察：10sbh 全是序列维上的 pointwise 运算 → 沿序列轴切 LayerNorm / Dropout。
- 前向：g 是 all-gather，ĝ 是 reduce-scatter；反向两者互换。
- 效果：激活显存也随机器数线性下降 → 显存真正线性扩展。

05 专家并行与其他并行

5.1 专家并行（expert parallel, EP）

- 不切 matmul，而是切专家、路由激活（all-to-all 分发 token）。
- 对 MLP 来说行为类似 TP：高带宽、减少激活；但切 matmul 可能比路由激活效率低。

5.2 EP 与其他并行的组合

- 朴素地可以组合所有策略，但注意：DP 通常与 EP 共享副本（所以 EP < DP）；DP 和 TP 可能相互作用降低利用率。

5.3 解耦 attention / expert 并行（Megatron）

- MoE 只作用于 MLP，不作用于 attention → attention 和 MLP 的并行可能失衡。
- 高 TP 利于 attention（不能用 EP）；低 TP 利于 MLP（更想用 EP 而不是 TP）。
- Megatron 方案：attention 用 TP/CP/DP，MLP 用 ETP/EP/EDP，各自独立。

5.4 Context parallel / Ring attention

- 长序列：把激活沿序列切到多卡（ring attention）。

5.5 并行方法总表（slide 55）

方法	通信 / 同步	每 rank 参数显存	激活 / KV 显存	主要带宽开销	扩全局 batch?	易用性
DDP / ZeRO-1	每步梯度 all-reduce	无参数分片（ZeRO-1 只分优化器状态）	无	梯度 ~O(params)	是，随 DP 线性	非常容易
FSDP / ZeRO-3	梯度 all-reduce（可重叠）	参数/梯度/优化器 ~1/DP	无	参数 ~O(params)，比 DDP 高，可重叠	是，随 DP 线性	中等
Pipeline	层间激活、流水线气泡	~1/PP	取决于流水线 buffer	stage 间激活流量	否，但需要 micro-batch	难

方法	通信 / 同步	每 rank 参数显存	激活 / KV 显存	主要带宽开销	扩全局 batch?	易用性
Tensor	阻塞式激活通信	TP 分片权重 ~1/TP	相关 matmul 激活 ~1/TP（配 SP）	每 block 激活级集合通信	否	难
Sequence / Context	每层序列分片交换	无	序列侧激活 / KV ~1/SP 或 1/CP	激活 / KV 通信	否	难
Expert (MoE)	每 MoE token dispatch all-to-all	仅专家权重 ~1/EP	无	token 路由 all-to-all	否，但每专家需足够 token	难

06 组合策略与实战案例

6.1 3D（4D）并行：组合规则

文献里的简单规则（simple rules of thumb）：

- 规则 1（直到模型装进显存）：TP / EP 扩到每机 GPU 数；PP 跨机器（或按带宽用 ZeRO-3）。
- 规则 2（然后直到 GPU 用完）：剩余全部用 DP。
- batch 小 → 梯度累积（gradient accumulation），用更大有效 batch 换通信效率。
- 模型并行 vs 张量并行（TPU book）：关键量是全局 batch size（除以 GPU 数）。

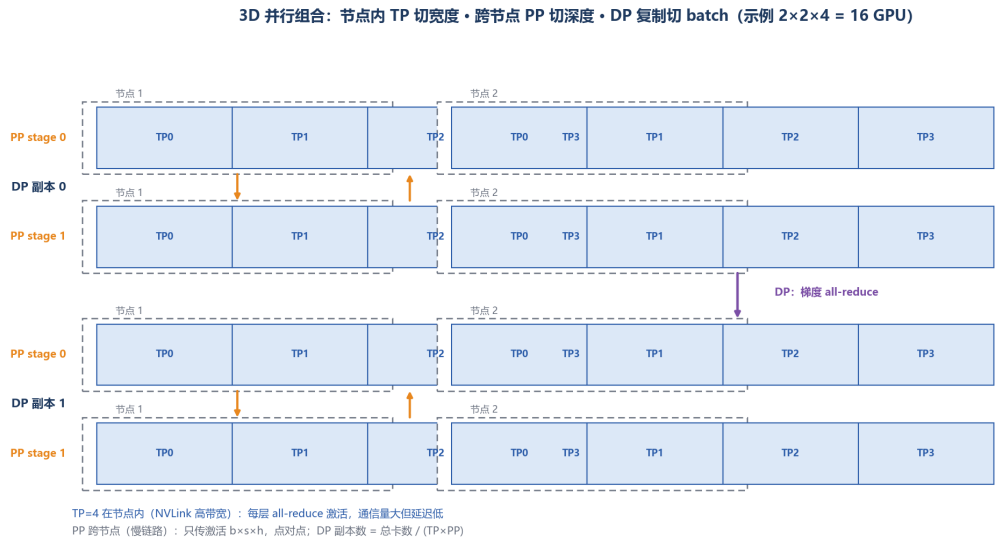


图 5 3D 并行组合：节点内 TP 切宽度 · 跨节点 PP 切深度 · DP 复制切 batch（示例 2×2×4 = 16 GPU）

6.2 经验法则

- Narayanan 2021：TP 先升到 8 然后封顶；PP 升到模型装下为止；DP 随规模逐渐减小，最大模型 DP=6。
- 精心设计的 3D 并行 → 线性收益，利用率保持平坦。

- TP=8 常常最优：跨 64 台机器时，8×8 配置最好。
- 激活重计算可以"回本"：省下的显存允许更大 batch，提升吞吐（t=8, p=16 的例子）。

6.3 真实模型怎么配

- Dolma 7B：FSDP（大概率节点内装得下）。
- DeepSeek V3：ZeRO-1 + TP + SP + PP(16) + EP(64-way, 8 节点)；EP 用 1F1B A2A overlap。
- Yi：ZeRO-1 + TP + PP；Yi-lightning (2025)：TP 换成 EP。
- Llama3 405B：三阶段（小 batch 训练 / 预训练 / 长上下文）；DP/TP/EP/PP/CP = 128/8/0/16/1；侧记：这个规模 GPU 故障很多。
- Gemma 2（2/9/27B）：ZeRO-3 + MP(=TP+SP) + DP。
- Mixtral 8x22B（Megatron 文档）：TP/PP/CP/EP = 4/4/1/8，DP 大概 2（凑 256 GPU）。
- Nemotron 3 Super 120B-A12B（长上下文扩展）：TP/PP/CP/EP = 2/0/64/64。
- Qwen3（225B-A22B 和 30B-A3B）：主要 EP 到 8 卡，大模型 2/8/32 并行。

6.4 案例总表（slide 72）

模型	DP	TP/SP	EP	PP	CP
DeepSeek	?? (ZeRO-1)	1	8	16	??
DeepSeek V3	?? (ZeRO-1)	1	64	16	??
Yi	?? (ZeRO-1)	>0	1	>0	??
Llama3 405B	128	8	0	16	1
Gemma 2	768	8	0	0	0
Mixtral 8x22 (megatron)	2	4	8	4	1
Nemotron 3 120B (long context)	??	2	64	??	64
Qwen 3 (megatron)	??	2	32	8	1

规律
TP 一般 ≤ 8；EP 可以很大（但难）；长上下文阶段用大 CP。

07 全讲回顾

- 超过某个规模，必须多 GPU、多节点并行。
- 并行问题没有单一解（大概三种思路都要用）。
- 组合不同并行形式有简单、可解释的经验法则。

一句话收尾

按带宽分层：节点内 TP/EP（高带宽、每层通信），跨节点 PP（点对点、传激活），剩余 DP（all-reduce 梯度）；显存不够再上 ZeRO-3 / SP，batch 不够就梯度累积。